

CodeSentinel AI

Autonomous Python Security & Refactoring Agent

Project Name	Section
CodeSentinel AI	B
Student Name	Seat No.
Muskan Shahid	23122094
Submitted To	Department
Miss Uzma	Artificial Intelligence

→

→

1. PROBLEM FORMULATION

Problem: Python programs may contain security vulnerabilities such as dangerous `eval()/exec()`, unsafe subprocess commands, and hardcoded passwords or API keys.

Goal: The goal of CodeSentinel AI is to detect security vulnerabilities, identify their risk level, explain the problems, generate safer code, and verify the fixed code.

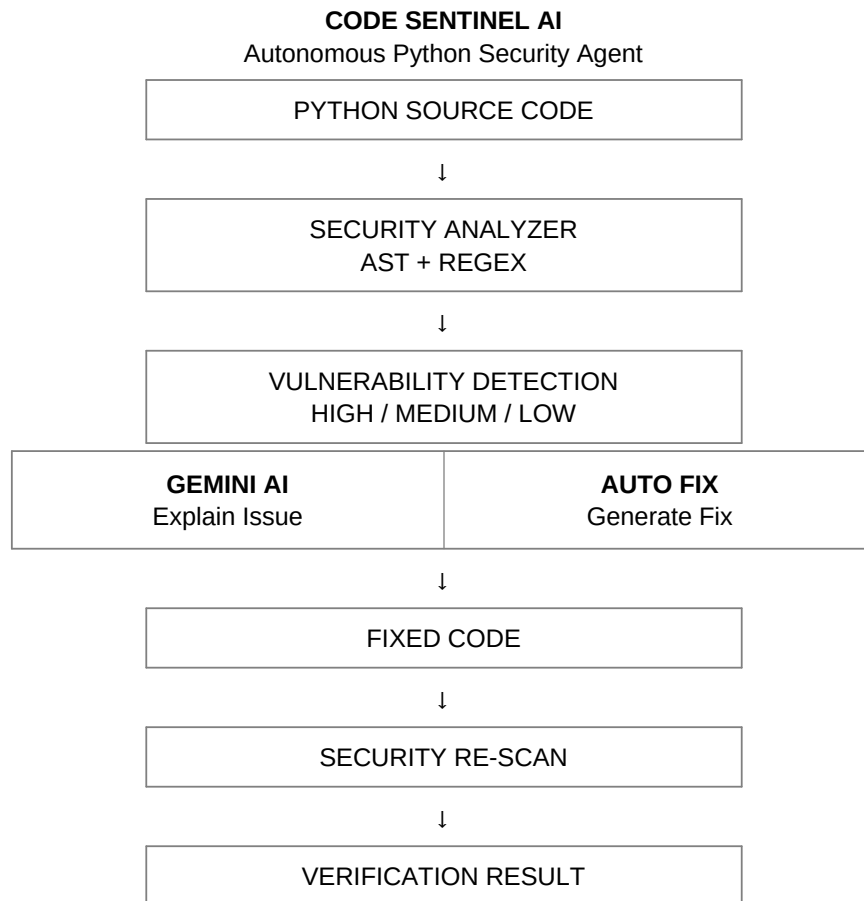
Input: Python Source Code

Verification Result

Output: Security Findings → Explanation Fixed Code

Agent: CodeSentinel AI **Environment:** Python source files and software development environment

2. GRAPHICAL REPRESENTATION / ENVIRONMENT



Environment Interaction

Python Code Environment → Source Code / Files → **CodeSentinel AI Agent** → Security Report, Explanation, Fixed Code, Verification Result

3. AGENT FORMULATION

Component	Description
Agent	CodeSentinel AI
Goal	Detect, explain, fix, and verify Python vulnerabilities
Environment	Python source code and software development environment
Sensors	AST parser, Regex rules, source-code input
Actuators	Security reports, explanations, fixed code, verification results

4. PEAS EVALUATION

P — Performance Measure	Accurate detection, correct risk classification, safe fixes, and successful verification
E — Environment	Python source files, codebases, and Streamlit interface
A — Actuators	Generate explanations, fixed code, reports, and verification results
S — Sensors	AST parser, Regex matching, and source-code analysis

Performance Evaluation: Detection accuracy, risk classification, quality of generated fixes, reduction of vulnerabilities, successful re-scanning, and clear security explanation.

5. ENVIRONMENT PROPERTIES

Property	Classification	Reason
Observability	Fully Observable	The agent receives the source code required for analysis
Determinism	Mostly Deterministic	Static analysis is predictable, while AI-generated responses may vary
Episodic / Sequential	Sequential	Fixing and verification depend on previous analysis results
Static / Dynamic	Static	The code remains unchanged during analysis unless modified by the agent
Number of Agents	Single-Agent	CodeSentinel AI performs the main intelligent tasks
Discrete / Continuous	Discrete	Code, vulnerabilities, risk levels, and actions are discrete
Known / Unknown	Known	The analysis rules and available actions are defined

6. WHY CODESENTINEL AI IS AN AI AGENT

CodeSentinel AI acts as an intelligent agent because it:

Perceives → Analyzes → Decides → Acts → Verifies

It receives Python code through its sensors, analyzes the code for vulnerabilities, identifies security problems, generates appropriate fixes, and verifies the fixed code through re-scanning.

7. TECHNOLOGIES USED

Python • AST • Regex • Google Gemini AI • Streamlit • Bandit • GitHub

8. CONCLUSION

CodeSentinel AI is an autonomous Python security agent designed to detect, explain, fix, and verify common security vulnerabilities. It combines static code analysis with AI-powered assistance to help developers identify and address security issues efficiently.

DETECT → EXPLAIN → FIX → VERIFY